

README

Part 11: 对齐实战 — verl 工业级 RL 后训练

- > Part 8 手写了 PPO/GRPO 的原理（玩具规模）；本部分把它们放上工业级 RL 基建 verl
- > （字节跳动 HybridFlow，DAPO/Seed-Thinking/Doubao 的训练系统），在真实 0.5B 模型上跑
- > GRPO。学完你能承担"跑 RL 实验、改奖励函数、调 rollout 配置"这类真实的对齐岗日常。
- > 主源：[verl-project/verl](https://github.com/verl-project/verl)（23.2k，Apache-2.0）

学习目标

完成本部分后，你将能够：

- 理解 RL 后训练在 LLM 链路中的位置和价值
- 手写 RLVR 奖励函数（\boxed / #### / 最后数字的抽取链）并解释 GRPO"全对组优势全零"现象
- 画出 从手写 GRPO 到 verl 三角色架构的映射图
- 配置 verl 的 PPO/GRPO 训练并读懂关键日志
- 识别 reward hacking 的风险并设计防范策略

章节导航

	内容
PPO 到 verl：概念桥接](01_handwritten_to_verl.md)	奖励函数/组内优势/KL 三零件手写（`01`）+ 装配成会学习的玩具 GRPO 训练循环（`02`）+ 逐概念映射
手：0.5B GRPO 实战](02_verl_quickstart.md)	Docker 环境 → quickstart PPO → 换 GRPO → 自定义奖励 → 双卡

前置知识

- 必须掌握：
- Part 8 04 章：手写 PPO（GAE/clip）与 GRPO（组内标准化）——本章的"手写侧"
 - Part 8 07 章：评估学（RLVR 的"可验证奖励"就是它的应用）

- 建议掌握：
- Part 10：FSDP 与多卡基础（verl 的训练后端就是 FSDP2/Megatron）
 - Part 8 02 章：SFT 训练流程（RL 是 SFT 之后的阶段）

- 可选：
- Part 14：vLLM 推理引擎（verl 的 rollout 引擎用 vLLM/SGLang）

在 LLM 链路中的位置

预训练(Part 7) → SFT(Part 8/12) → 【本部分: RL 后训练 (PPO/GRPO/DAPO…)】 → 部署(Part 14)
↑
你在这里

为什么这一步是 2026 年的主战场：

证据	说明
GLM-5.3	基座与 5.2 完全相同，全部提升来自后训练 RL Scaling
DeepSeek-V4	把 GRPO 下沉到"专家模型"层
Kimi-Researcher	平均 23 次工具调用/回答，端到端 RL on hard tasks

框架生态：verl（字节）与 slime（智谱，8.3k）是两大开源 RL Scaling 框架。

环境与版本策略 (⚠️ 全课程安装摩擦最高的一章)

verl 的 vllm/torch/transformers 版本锁步耦合严重——用官方 Docker，不要裸 pip：

`bash`

策略：latest release tag 的官方镜像（锁版本 + 免依赖地狱）

`docker pull verlai/verl:latest`

最小硬件：官方 quickstart = Qwen2.5-0.5B 单卡
PPO（文档明确 ≥24GB）

→ 单张 4090 可跑（micro-batch 置 1 防 OOM）；双卡可玩 FSDP 分片

你有什么	能做什么
CPU only	01 章脚本 01/02（纯 Python 三零件 + 玩具 GRPO 训练循环，均 CPU 可跑）+ 通读概念
1 × 4090	Docker quickstart 完整跑通（0.5B PPO/GRPO，小时级）
2 × 4090	+ FSDP 分片实验（02 章第 5 步）

学习地图

手写过的 GRPO（Part 8） ← 点（原理）
↓ 逐概念映射
奖励函数手写（脚本01：RLVR 入口）
↓
玩具 GRPO 训练循环（脚本02：三零件装配，会学习）
↓
verl quickstart 跑通 ← 线（工业工具）
↓ GRPO / 自定义奖励 / 双卡
读 recipe/dapo（生产配方） → 面试/工作就绪

课后作业

每章末尾有思考题（<details> 折叠答案）。全部学完后：

[Assignment 11](../assignments/assignment_11/)

相关资源

- [verl](https://github.com/verl-project/verl) (docs/quickstart 是最接近教程的官方材料)
- [slime](https://github.com/THUDM/slime) (智谱 RL Scaling 框架, GLM-4.5 起的 GLM 系列用)
- [rasbt/reasoning-from-scratch](https://github.com/rasbt/reasoning-from-scratch) (裸 PyTorch 从零写 RLVR-GRPO——原理侧对照)
- HybridFlow (EuroSys'25) · GRPO (arXiv 2402.03300) · DAPO

[← 上一章：Part 10 分布式](../Part10_distributed/tutorial/README.md) | [下一章：Part 12 LLaMA-Factory →](../Part12_finetune_llamafactory/tutorial/README.md)

01_handwritten_to_verl

01 — 从手写 GRPO 到 verl：概念桥接

- > 你在 Part 8 已经手写过 GRPO：采样 G 个回答 → 打分 → 组内标准化优势 → clip 更新。
- > verl 把同一套数学放进工业引擎：rollout 用 vLLM/SGLang、训练用 FSDP2、编排用 Ray。
- > 本章先手写并验证这条管线的三件零件（[脚本 01](../scripts/01_reward_and_bridge.py)），
- > 再装配成一个会学习的玩具训练循环（[脚本 02](../scripts/02_grpo_toy_train.py)），
- > 最后给出逐概念的"手写 ↔ verl"映射表——02 章进 Docker 时你不会迷路。

学习目标

完成本章后，你将能够：

- 手写 RLVR 奖励函数（\boxed / ##### / 最后数字的抽取链）
- 解释 GRPO"全对组优势全零"现象及其数学原因
- 画出"手写 GRPO 循环 → verl 三角色 + 权重同步"的映射图
- 说清 RL 训练为什么需要两个引擎（rollout + training）

前置知识

必须掌握：

- Part 8 04 章：GRPO 的组内优势与 KL 惩罚（本章的直接上游）
- Part 8 07 章：评估学（RLVR = 用评估学里的"规则评估"当奖励）

建议回顾：

- Part 8 02 章：SFT 训练流程（RL 是 SFT 之后的阶段）

理论背景

问题引入：为什么 SFT 之后还需要 RL？

SFT（Supervised Fine-Tuning）教会模型"模仿"人类示范，但有两个根本限制：

- 1. 覆盖率问题：人类示范只能覆盖一小部分可能的输入空间
- 2. 优化目标问题：SFT 优化的是"模仿人类"，而非"解决问题"

RL 后训练（RLHF/GRPO/DAPO）通过试错学习来弥补：

SFT: "看人类怎么做，模仿它" → 学会格式和风格
RL: "自己试，看结果好坏，改进" → 学会推理和决策

- > 类比：SFT 像是看教学视频学游泳，RL 像是自己下水练习。教学视频教你动作，
- > 但只有实际练习才能让你真正学会游泳。

数学推导：GRPO 的组内优势

GRPO（Group Relative Policy Optimization）的核心思想是：**用同一 prompt 的多个回答相互比较，而非依赖单独的 Value 网络**。

问题设定：

- 给定一个 prompt，生成 G 个回答： $\{y_1, y_2, ..., y_G\}$
- 每个回答有一个奖励： $\{r_1, r_2, ..., r_G\}$

推导过程：

Step 1: 计算组内均值

$$\text{mean} = (1/G) * \sum_{i=1}^G r_i$$

Step 2: 计算组内标准差

$$\text{std} = \sqrt{(1/G) * \sum_{i=1}^G (r_i - \text{mean})^2}$$

Step 3: 计算优势值（advantage）

$$A_i = (r_i - \text{mean}) / \text{std}$$

性质：

- $\sum A_i = 0$ （优势之和为零）
 - 如果所有 r_i 相同，则 $\text{std} = 0$ ，所有 $A_i = 0$
- "太简单的题没有梯度"，GRPO 天然跳过已掌握样本

与 PPO 的对比：

维度	PPO	GRPO
基线来源	Value 网络（需要训练）	组内均值（不需要训练）
显存开销	需要额外的 Value 网络	无额外网络
稳定性	Value 网络可能不稳定	组内比较更稳定
梯度效率	每个样本都有梯度	全对/全错组无梯度

- > 关键洞察：GRPO 的"全对组优势全零"不是 bug，而是 feature——
- > 它天然跳过已掌握的样本，把计算资源集中在需要学习的样本上。

历史脉络：PPO → GRPO → DAPO

2017: PPO (OpenAI)

↓ 简化 Value 网络

2024: GRPO (DeepSeek)

↓ 解决全对/全错组无梯度问题

2025: DAPO (字节跳动)

↓ 工程优化 (clip-higher, token-level loss)

2025: verl (字节跳动)

↓ 工业级 RL Scaling 框架

关键论文：

- PPO: [Proximal Policy Optimization Algorithms](https://arxiv.org/abs/1707.06347)
- GRPO: [DeepSeekMath: Pushing the Limits of Mathematical Reasoning](https://arxiv.org/abs/2402.03300)
- DAPO: [DAPO: An Open-Source LLM Reinforcement Learning System](https://arxiv.org/abs/2503.14476)

代码实现

手写三件套（脚本 01，CPU 可跑）

运行 [scripts/01_reward_and_bridge.py](../scripts/01_reward_and_bridge.py) 验证以下代码。

① 可验证奖励函数（RLVR 的核心）

```
python
def gsm8k_reward(response: str, ground_truth: str) -> float:
    """从模型回答里抽取数字，对则 1 分否则 0 分。
```

抽取顺序（工程惯例）：\\boxed{} 优先 → '#### 42' 标记 → 最后一个数字

数据流：

response(str) → 正则匹配 → pred(str) → float(pred) → 比较 → reward(float)

常见陷阱：

- response 为空字符串 → 返回 0.0
- ground_truth 包含千分位逗号 "1,234" → 需要处理
- 浮点数精度问题 → 用 $abs(pred - gt) < 1e-4$ 判断

```
"""
```

Step 1: 尝试抽取 \\boxed{} 中的内容

```
m = re.findall(r"\\boxed\\{(?:[\\d\\.]+)\\}", response)
```

Step 2: 如果没有 \\boxed{}，尝试抽取 '#### 42' 格式

```
if not m:
```

```
m = re.findall(r"####s*(-?[\d,\.]+)", response)
```

Step 3: 如果都没有，抽取最后一个数字

```
if not m:  
m = re.findall(r"-?\d+\.?\d*", response.replace(",", ""))
```

Step 4: 没有找到任何数字 → 错误

```
if not m:  
return 0.0
```

Step 5: 取最后一个匹配的数字

```
pred = m[-1].replace(",", "").rstrip(".")
```

Step 6: 比较预测值和真实值

```
try:  
return 1.0 if abs(float(pred) - float(ground_truth)) < 1e-4 else 0.0  
except ValueError:  
return 0.0  
,
```

实测输出（脚本 01, Python 3.12 / CPU）：

[1] GSM8K 规则奖励函数（verl quickstart 的 custom reward 同语义）：
数据流: response(str) → reward(float)

```
'答案是 \boxed{42}。' gt= 42 → reward=1.0  
'#### 3.5' gt= 3.5 → reward=1.0  
'我觉得是 100，不对，是 7。' gt= 7 → reward=1.0  
'答案 \boxed{41}' gt= 42 → reward=0.0  
'我不会。' gt= 42 → reward=0.0  
'1,234 个。' gt= 1234 → reward=1.0  
,
```

② 组内优势（GRPO 核心）

```
`python  
def group_advantages(rewards_per_prompt, eps=1e-6):  
    """每个 prompt 采 G 个回答 → A_i = (r_i - mean) / std。
```

数学推导：

```
mean = (1/G) * Σ r_i  
std = sqrt((1/G) * Σ (r_i - mean)^2)  
A_i = (r_i - mean) / std
```

性质：

- $\Sigma A_i = 0$ （优势之和为零）
 - 如果所有 r_i 相同，则 $std = 0$ ，所有 $A_i = 0$
- "太简单的题没有梯度"

数据流：

```
rewards(n_prompts, n_responses) → advantages(n_prompts, n_responses)
```

常见陷阱：

- 组内全对（全 1.0）→ std = 0 → 优势全 0（无梯度）
- 组内全错（全 0.0）→ std = 0 → 优势全 0（无梯度）

```
"""
```

```
advs = []
```

```
for group in rewards_per_prompt:
```

```
    r = group
```

```
    n = len(r)
```

Step 1: 计算组内均值

```
mean = sum(r) / n # shape: scalar
```

Step 2: 计算组内标准差

```
var = sum((x - mean) ** 2 for x in r) / n # shape: scalar
```

```
std = max(var ** 0.5, eps) # shape: scalar, 防止除零
```

Step 3: 计算优势值

```
group_adv = [(x - mean) / std for x in r] # shape: (n_responses,)
```

```
advs.append(group_adv)
```

```
return advs
```

实测输出（脚本 01）：

```
,
```

[2] 组内优势（adv_estimator=grpo 的语义）：

数据流: rewards(n_prompts, n_responses) → advantages(n_prompts, n_responses)

prompt0 rewards=[1.0, 0.0, 1.0, 0.0] → adv=[1.0, -1.0, 1.0, -1.0]

prompt1 rewards=[1.0, 1.0, 1.0, 1.0] → adv=[0.0, 0.0, 0.0, 0.0]

prompt2 rewards=[0.0, 0.0, 1.0, 0.0] → adv=[-0.58, -0.58, 1.73, -0.58]

⚠ 关键观察：prompt1 全对 → 优势全 0：'太简单的题没有梯度'

这是 GRPO 的天然特性：已掌握的样本不会产生梯度更新

```
,
```

> 手算验证（prompt0）：mean = 0.5, std = sqrt((0.25×4)/4) = 0.5,

> 所以 A = ±0.5 / 0.5 = ±1.0。注意这里是 std 归一化（除以组内标准差），

> 不是除以均值，也不是 RMS——GRPO 论文原式即此；adv=[0.71, ...] 一类数值

> 出自别的归一化口径，读别的实现时留意分母是什么。

③ k3 KL 估计器

```
`python
```

```
def k3_kl(logp_ref, logp_new):
```

```
    """KL(π_new || π_ref) 的低方差估计：exp(d) - d - 1, d = logp_ref - logp_new
```

数学推导：

$$\text{KL}(q \parallel p) = \mathbb{E}_q[\log(q/p)] = \mathbb{E}_q[\log q - \log p]$$

令 $d = \log p_{\text{ref}} - \log p_{\text{new}}$

则 $KL = E[\exp(d) - d - 1]$

性质：

- $\exp(d) - d - 1 \geq 0$ 对所有 d 成立（因为 $e^x \geq x + 1$ ）
- 当 $d = 0$ 时取等号（两个分布相同）

数据流：

`logp_ref(list), logp_new(list) → kl(scalar)`

"""

`kl = 0.0`

`for lr, ln in zip(logp_ref, logp_new):`

`d = lr - ln # shape: scalar`

`kl += math.exp(d) - d - 1`

`return kl / len(logp_ref)`

`

实测输出（脚本 01）：

`

[3] k3 KL (verl 的 KL 惩罚形态)：

数据流: `logp_ref(list), logp_new(list) → kl(scalar)`

$KL(\pi_{\text{new}} \parallel \pi_{\text{ref}}) = 0.0204$

性质: ≥ 0 （恒非负，估计器保证）

`

> 手算验证： $d_1 = \log(0.4) - \log(0.5) = \log(0.8)$, $\exp(d_1) - d_1 - 1 =$

$> 0.8 + 0.2231 - 1 = 0.0231$ ； $d_2 = \log(1.2)$, $\exp(d_2) - d_2 - 1 = 1.2 - 0.1823 - 1 =$

> 0.0177 ；平均 $= (0.0231 + 0.0177) / 2 = 0.0204$ 。

从零件到训练循环（脚本 02，CPU 可跑）

上面三件是"零件"；`[scripts/02_grpo_toy_train.py](../scripts/02_grpo_toy_train.py)`

把它们装配成一个真正会学习的 GRPO 训练循环——玩具任务：6 道猜数字题

（候选 0-3），小策略模型（[Embedding\(6,16\)](#) → [Linear\(16,4\)](#)），每题采 $G=4$ 个回答，回答拼成 `\boxed{d}` 字符串后走 ① 的同一条奖励链打分。核心循环：

``python`

`for step in range(N_STEPS):`

rollout：策略采样 G 个回答（= verl 的 rollout 角色）

`actions, logp, rewards = rollout_and_score(policy, prompt_ids, targets, G)`

actions/logp/rewards shape: (G, P) ，每列是一道题的组

advantage：组内标准化（= adv_estimator=grpo）

`groups = rewards.t().tolist() #  $(P, G)$ ：每行一个组`

`adv_t = torch.tensor(group_advantages(groups)).t() # 转回  $(G, P)$  对齐 logp`

`skip_ids = zero_adv_groups(groups) # 全对/全错组 → 本轮无梯度`

KL：冻结的 ref 策略打分，k3 估计器进 loss（= ref 角色 + kl_penalty）


```

with torch.no_grad():
    ref_logp = torch.distributions.Categorical(
        logits=ref_policy(prompt_ids)).log_prob(actions) # (G, P)
    d_t = ref_logp - logp # 可反传
    kl_pen = (d_t.exp() - d_t - 1).mean() # k3 的 torch 形态

```

loss & 更新：零优势项自动无贡献 → 零梯度组被天然跳过

```

loss = -(logp adv_t).mean() + BETA kl_pen
opt.zero_grad()
loss.backward()
opt.step()

```

实测输出（脚本 02，Python 3.12 / CPU，固定种子 42，约 1-2 秒）：

任务：6 道猜数字题（候选 0-3），每题采 G=4 个回答
 配置：steps=60, lr=0.5, KL 系数 $\beta=0.02$

[1] GRPO 训练循环（rollout → 规则奖励 → 组内优势 → KL 惩罚 → 更新）：
 数据流: prompts(P,) → logits(P,V) → actions(G,P) → rewards(G,P) → advs(G,P)

step 0: 平均奖励=0.38 零梯度组=1/6（全对0+全错1） KL=0.000
 step 4: 平均奖励=0.67 零梯度组=6/6（全对4+全错2） KL=0.413
 step 9: 平均奖励=0.83 零梯度组=6/6（全对5+全错1） KL=0.341
 step 14: 平均奖励=0.83 零梯度组=6/6（全对5+全错1） KL=0.341
 step 29: 平均奖励=0.83 零梯度组=6/6（全对5+全错1） KL=0.341
 step 44: 平均奖励=0.83 零梯度组=6/6（全对5+全错1） KL=0.341
 step 59: 平均奖励=0.83 零梯度组=6/6（全对5+全错1） KL=0.341

零梯度组数量变化（每 10 步）: [1, 6, 6, 6, 6, 6]

[2] 零梯度组现场（训练后策略变自信；全对组=已掌握，全错组=卡死，优势都全 0）：
 prompt0 rewards=[0.0, 0.0, 0.0, 0.0] → adv=[0.0, 0.0, 0.0, 0.0] ← 零梯度组（跳过）
 prompt1 rewards=[1.0, 1.0, 1.0, 1.0] → adv=[0.0, 0.0, 0.0, 0.0] ← 零梯度组（跳过）
 prompt2 rewards=[1.0, 1.0, 1.0, 1.0] → adv=[0.0, 0.0, 0.0, 0.0] ← 零梯度组（跳过）

(a) 平均奖励: 初始 0.38 → 最终 0.83

[3] BC（行为克隆）基线：同样的模型与步数，直接用标准答案做交叉熵：

step 0: 准确率=0.50
 step 14: 准确率=1.00
 step 29: 准确率=1.00
 step 44: 准确率=1.00
 step 59: 准确率=1.00

三个关键观察（脚本 02 的全部意义所在）：

- (a) 平均奖励上升：0.38 → 0.83。奖励只来自规则验证器——模型从头到尾没见过任何标准答案标签，这就是 RLVR 的"以验证代替标注"。
- (b) 零梯度组的两种命运：**prompt1/2** 全对 = 已掌握，跳过是 feature（算力集中在有区分度的题上）；**prompt0** 全错 = 卡死，跳过是盲区——它永远学不会，最终停在 0.83 而非 1.00。DAPO 的 dynamic sampling（把全对/全错组过滤掉重新采样）正是专门治这个病。
- (c) BC 基线对比：同样模型、同样步数，BC 用标签做交叉熵直接到 1.00——

有标签时监督学习是上限；GRPO 的价值在于没有标签、只有验证器的场景（数学对错、代码单测、格式校验）。

- > 看懂这个循环 = 看懂 verl 配置：rollout_and_score →
- > actor_rollout_ref.rollout, math_reward → custom reward function,
- > group_advantages → adv_estimator=grpo, ref_policy + k3 → ref 角色 +
- > algorithm.kl_penalty。02 章的每一行配置，在本节都有对应的手写代码。

工程实践

为什么工业版必须"两个引擎"

手写版玩具模型 1 秒能生成 100 个回答；真实 7B 模型生成一个回答要几百 ms——rollout 占 RL 训练时间的大头（典型占比：rollout 60-80%、reward 5-10%、training 15-30%，完整分布表见 [02 章"性能分析"]([02_verl_quickstart.md#性能分析](#))）。

解决方案：分离 rollout 和 training 引擎

引擎	用途	技术选型
rollout 引擎	高吞吐生成	vLLM / SGLang
training 引擎	高效训练	FSDP2 / Megatron

代价：每次更新后要把新权重搬进推理引擎（大模型上这是 GB 级拷贝）。

verl 的优化：HybridEngine 用重分片+原地转换把这一步的开销压到最低。

常见陷阱

陷阱 1：Reward Hacking

症状：模型学会"钻空子"获得高奖励，但并没有真正解决问题

原因：奖励函数有漏洞，模型找到了"作弊"方式

示例：

```
python
```

不好的奖励函数：只看最后数字

```
def bad_reward(response, ground_truth):
    pred = extract_last_number(response)
    return 1.0 if pred == ground_truth else 0.0
```

模型可能学会：不管问题是什么，总是输出 "42"

因为某些问题的答案恰好是 42

,

解法：

- 奖励函数要尽可能严格（检查格式、检查推理过程）

- 添加惩罚项（如回答过长、格式错误）
- 使用多个奖励函数组合

陷阱 2：全对/全错组无梯度

症状：训练一段时间后，loss 不下降

原因：所有 prompt 的所有回答都对（或都错），优势全为 0

解法：

- 使用课程学习（Curriculum Learning）：从简单到难
- 过滤已掌握的样本（全对的题跳过）
- 增加组大小 n （更多采样，更可能有区分度）

陷阱 3：版本冲突

一句话：verl 与 vlm/torch/transformers 版本锁步耦合，用官方 Docker 镜像、不要裸 pip——完整症状与解法见 [02 章"陷阱 2: 版本冲突"]([02_verl_quickstart.md#陷阱-2-版本冲突](#))。

最佳实践

奖励函数设计

1. 规则优先：能用规则判断的就用规则（RLVR）
2. 多维度评估：正确性 + 格式 + 推理过程
3. 防作弊：检查回答是否"真正"解决了问题
4. 可解释：奖励函数的逻辑要清晰，便于调试

配置调优

1. 组大小 n ：常用 4-16，越大越稳定但越贵
2. KL 惩罚系数：防止策略偏离太远，常用 0.01-0.1
3. 学习率：RL 阶段通常比 SFT 阶段小
4. micro-batch：根据显存调整，4090 上通常置 1

调试展示：常见错误与修复

错误 1：奖励函数返回 None

症状：

```
`python
reward = gsm8k_reward("答案是 42", "42")
print(reward) # None
`
```

原因：函数没有 return 语句，或者 return 语句在 if 分支里但没有 else

解法：

```
`python
def gsm8k_reward(response, ground_truth):
```

... 抽取逻辑 ...

```
if not m:
    return 0.0 # 必须有 return, 不能只是 pass
```

... 比较逻辑 ...

```
return 1.0 if match else 0.0 # 确保所有路径都有 return
```

错误 2：组内优势出现 NaN

症状：

```
`python
adv = group_advantages([1.0, 1.0, 1.0, 1.0])
print(adv) # [nan, nan, nan, nan]
```

原因：全同组的 $\text{std} = 0$ ，导致除零

解法：

```
`python
def group_advantages(rewards, eps=1e-6):
    ...

    std = max(var ** 0.5, eps) # 使用 eps 防止除零

    ...
```

错误 3：KL 散度为负数

症状：

```
`python
kl = k3_kl([math.log(0.5)], [math.log(0.3)])
print(kl) # -0.1 (错误！KL 应该  $\geq 0$ )
```

原因：公式写错，应该是 $\exp(d) - d - 1$ 而不是 $d - \exp(d) + 1$

解法：

```
`python
def k3_kl(logp_ref, logp_new):
    for lr, ln in zip(logp_ref, logp_new):
        d = lr - ln
        kl += math.exp(d) - d - 1 # 注意顺序：exp(d) - d - 1
    return kl / len(logp_ref)
```

错误 4：verl Docker 启动失败

症状：

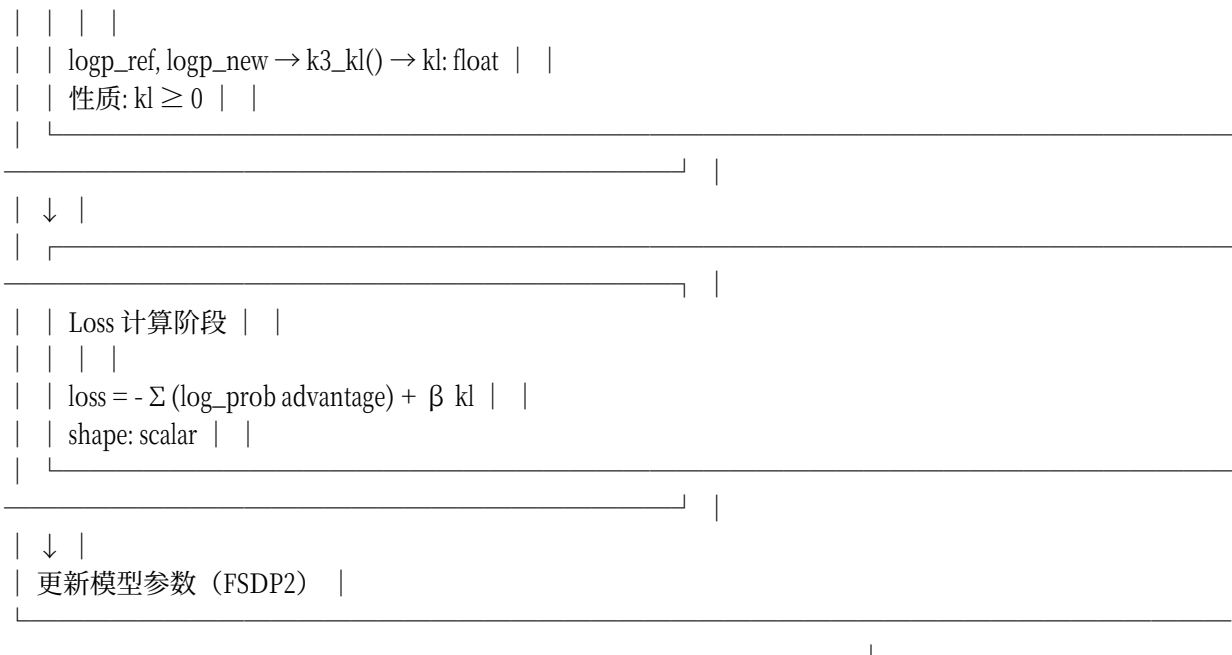
```
bash
docker: Error response from daemon: could not select device driver "nvidia"
```

原因：未安装 NVIDIA Container Toolkit

解法：安装 NVIDIA Container Toolkit 即可——完整安装命令在 [02 章“调试展示”的错误 1] (02_ver_quickstart.md#错误-1-docker-容器无法访问-gpu)，本章不重复（01 章的脚本不需要 Docker，CPU 就能跑）。

形状追踪：数据流全景图





性能数据（摘要）

RL 训练每步开销的量级（完整表见 [02 章"性能数据"]([02_verl_quickstart.md#性能数据量级参考](#)))：

- 0.5B GRPO @ 1×4090 (n=5)：每步 ~2s、显存 ~8GB
- 组大小 n 5→16：每步 ~2s → ~5s，显存 ~8GB → ~12GB
- 7B @ 2×4090：每步 ~30s、~20GB/卡，需要 QLoRA

> 量级来源：官方 benchmark 与课程设计推算（非本机实录；Docker 实操后请以自己日志为准）

手写 ↔ verl 概念映射表（本章核心产出）

你手写的（Part 8 04 章）	verl 里的对应	说明
玩具模型同时干生成+训练	**actor_rollout_ref 三个角色**	训练引擎（FSDP2/Megatron）与推理引擎（vLLM/SGLang）分离
`for step: 采样 G 个回答`	rollout 的 `generate_sequences`	大 batch 并行生成（这是 RL 训练最贵的阶段）
手动把新权重"告诉"生成器	**权重回同步（weight sync）**	3D-HybridEngine 消除 train↔rollout 转换的显存冗余（verl 的招牌）
`group_advantages()`	`adv_estimator=grp` 配置	数学一样；配置行替代你的 20 行
k3 KL / ref 模型	ref policy 角色 + KL 惩罚系数	ref 是 SFT 模型的冻结副本
`gsm8k_reward()`	**custom reward function**	quickstart 里你唯一必写的代码（RLVR 入口）
单进程 for 循环	**Ray 单控制器数据流**	工作器角色化、资源 placement 可编程

- 一句话理解 verl：它没有发明新算法——****算法（GRPO/PPO/DAPO…）是配置项，基建（rollout 引擎、权重同步、分布式）才是它的本体****。这就是"手写一遍再上工具"的价值：你看得懂配置背后的每一段代码在干什么。

学完本章你能...

- 手写 RLVR 奖励函数 (`\boxed{####}` / 最后数字的抽取链)
- 解释 GRPO"全对组优势全零"现象及其数学原因
- 画出"手写 GRPO 循环 $\rightarrow$ verl 三角色 + 权重同步"的映射图
- 说清 RL 训练为什么需要两个引擎 (rollout + training)
- 识别 reward hacking 的风险并设计防范策略

概念检验

<details>

<summary>Q1: 为什么规则奖励 (RLVR) 比训练一个奖励模型更受青睐? 什么时候不能用? </summary>

A: 优势:

- 不可作弊 (答案可机器验证)
- 无 RM 偏差 (奖励模型可能有偏见)
- 无 RM 被奖励黑客的风险

不能用的场景:

- 答案不可形式化的任务 (创意写作、对话质量)
- 需要主观判断的任务 ("这个回答有帮助吗?")
- 那才需要 RM/LLM-as-judge (Part 8 07 章)

</details>

<details>

<summary>Q2: 组内标准化的基线和 PPO 的 Value 网络基线各有什么问题? </summary>

A: GRPO 组内基线的问题:

- 同组样本太少时噪声大 (std 估计不准)
- 全对/全错组无梯度 (浪费算力)
- 可用课程难度 Curriculum 缓解

PPO Value 基线的问题:

- 要多训一个网络 (显存+不稳定性)
- Value 网络可能不准确
- 但能给单样本基线 (不需要组内比较)

DAPO/DrGRPO 等 recipe 在修这些边角 (02 章读 recipe/dapo)

</details>

<details>

<summary>Q3: 如果把组大小 n 从 4 提到 32, 成本和效果各怎么变? </summary>

A: 成本:

- rollout 成本线性 $\times 8$ (生成是 RL 最贵阶段)
- 显存也线性增加 (需要存储更多回答)

效果:

- 优势估计更准 (std 估计更稳)
- 更可能有"组内有区分度" (不会全对/全错)
- 但收益递减 (从 4 $\rightarrow$ 16 提升大, 从 16 $\rightarrow$ 32 提升小)

实际权衡: 在 8-16 之间权衡, 另配 prompt 难度过滤 (全对的题跳过)

</details>

动手实践

<details>

<summary>练习 1: 实现一个防作弊的奖励函数</summary>

任务：实现一个比 gsm8k_reward 更严格的奖励函数，检查答案正确性 + 推理过程。

验收标准：

- [] 检查答案正确性（使用 gsm8k_reward 的逻辑）
- [] 检查推理过程（至少有 "because", "therefore", "so" 等词）
- [] 惩罚过短的回答（< 10 字）
- [] 返回 0.0-1.0 之间的分数

步骤提示：

```
`python
def anti_hacking_reward(response: str, ground_truth: str) -> float:
```

Step 1: 检查答案正确性

```
answer_correct = gsm8k_reward(response, ground_truth)
```

Step 2: 检查推理过程

```
reasoning_words = ["because", "therefore", "so", "since", "thus"]
has_reasoning = any(word in response.lower() for word in reasoning_words)
```

Step 3: 检查回答长度

```
is_long_enough = len(response) >= 10
```

Step 4: 综合计算分数

```
if not answer_correct:
    return 0.0
if not has_reasoning:
    return 0.5 # 答案对但没有推理过程
if not is_long_enough:
    return 0.5 # 答案对但太短
return 1.0 # 答案对 + 有推理 + 足够长
```

</details>

<details>

<summary>练习 2: 实现一个 rollout 成本计算器</summary>

任务：实现一个函数，根据模型大小和组大小 n 计算 rollout 成本。

验收标准：

- [] 输入：模型参数量（B）、组大小 n、prompt 数量
- [] 输出：预估的 rollout 时间（秒）
- [] 考虑 GPU 数量和并行度

步骤提示：

```
`python
def estimate_rollout_cost(
    model_params_B: float, # 模型参数量（单位：B）
    n: int, # 组大小
    num_prompts: int, # prompt 数量
    num_gpus: int = 1, # GPU 数量
) -> float:
```



```
"""
```

估算 rollout 时间（秒）

经验公式：

- 每个 token 的生成时间 $\approx 0.1\text{ms} * \text{model_params_B}$
- 每个回答平均 100 tokens
- 总时间 = prompts n tokens * time_per_token / num_gpus

```
"""
```

TODO: 实现

```
pass
```

```
</details>
```

```
<details>
```

```
<summary>练习 3: 实现 KL 预算护栏</summary>
```

任务：实现一个函数，监控 KL 散度并在超预算时发出警告。

（教程选做：Assignment 11 的题 4 是它的简化版 `kl_budget_ok`——只返回 bool；这里的 `kl_budget_guard` 是带警告信息的完整版，作业测试不覆盖它。）

验收标准：

- [] 输入：logp_ref、logp_new、budget
- [] 输出：(是否在预算内, 当前 KL 值, 警告信息)
- [] 如果 $KL > budget$ ，返回详细的警告信息

步骤提示：

```
`python
def kl_budget_guard(
    logp_ref: list,
    logp_new: list,
    budget: float = 0.05,
) -> tuple:
    """
```

KL 预算护栏

Returns:

(is_ok, kl_value, warning_msg)

- is_ok: bool, 是否在预算内
- kl_value: float, 当前 KL 值
- warning_msg: str, 警告信息（如果超预算）

```
"""
```

TODO: 实现

```
pass
```

```
</details>
```

课后作业

完成本章后，去 Assignment 11 完成练习：

[Assignment 11](../../assignments/assignment_11/)

下一步

还没跑过脚本 02？先回去跑一遍——它把本章的三件套装进一个会学习的训练循环，是 02 章所有配置行背后的代码。然后进 Docker，把 0.5B 模型的 GRPO 真正跑起来。

[scripts/02_grpo_toy_train.py](../scripts/02_grpo_toy_train.py)（玩具 GRPO 训练循环，CPU 约 1-2 秒）
[02 — verl 快速上手：0.5B GRPO 实战](02_verl_quickstart.md)

02_verl_quickstart

02 — verl 快速上手：0.5B GRPO 实战

> 本章在 Docker 里跑通 verl 官方 quickstart（Qwen2.5-0.5B 在 GSM8K 上的 PPO/GRPO，
> 官方文档明确单卡 $\geq 24\text{GB}$ ——4090 直接可跑），然后换成 GRPO、写自己的奖励函数、
> 扩到双卡。所有步骤的“手写对应物”都标注 01 章的编号。

学习目标

完成本章后，你将能够：

- 配置 Docker 环境并启动 verl 容器
- 跑通 0.5B 模型的 PPO/GRPO 训练
- 读懂 verl 的关键日志（rollout/权重同步/优势计算）
- 编写自定义奖励函数并集成到 verl
- 扩展到双卡训练并理解 FSDP 分片

前置知识

必须掌握：

- 01 章：概念映射表（本章配置行 = 01 章的手写代码）
- Part 10：FSDP（第 5 步的扩展实验用）

建议回顾：

- Part 8 04 章：GRPO 的数学原理

理论背景

为什么需要 Docker？

verl 与 vllm/torch/transformers 版本锁步耦合严重：

,

verl 0.4.0 $\rightarrow$ vllm 0.8.x $\rightarrow$ torch 2.5.x $\rightarrow$ transformers 4.45.x

↓ 版本冲突
裸 pip install → 依赖地狱 → 无法运行

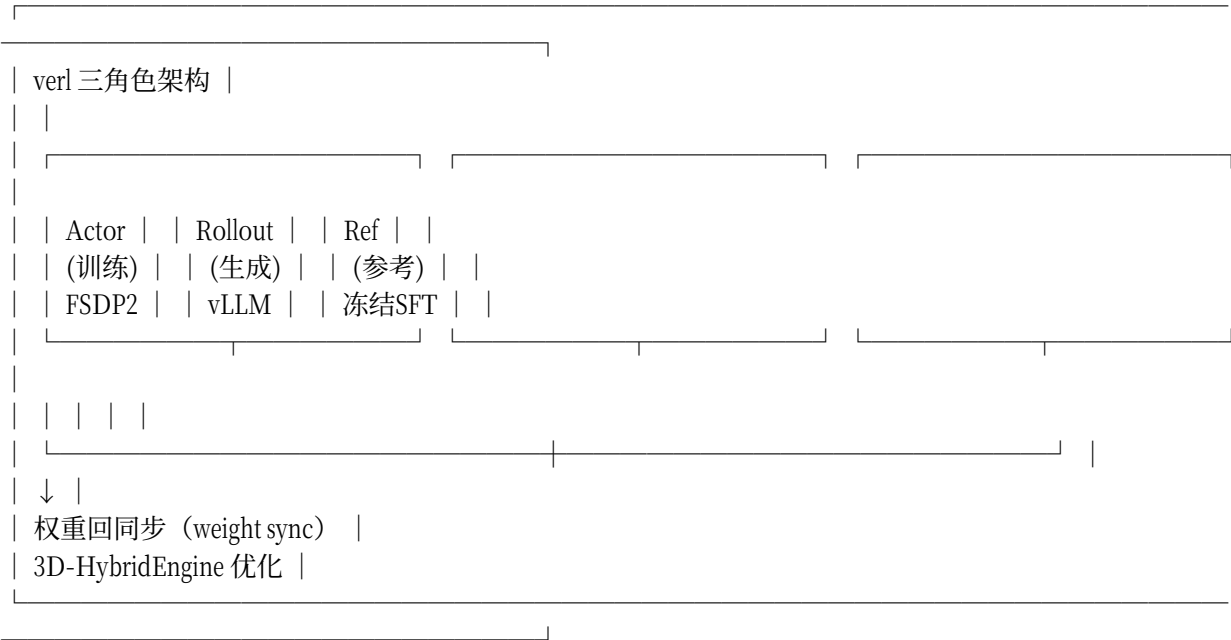
解决方案：官方 Docker 镜像锁定所有版本

```
bash
docker pull verlai/verl:latest
```

内含：verl + vllm + torch + transformers + 所有依赖

版本完全兼容，开箱即用

verl 的三角色架构



对应 01 章的手写代码：

verl 角色	手写对应	功能
Actor	训练循环	更新模型参数
Rollout	'for step: 采样 G 个回答'	生成回答
Ref	ref 模型	计算 KL 惩罚

代码实现

Step 1: 环境配置（Docker）

```
`bash
```

拉取官方镜像

```
docker pull verlai/verl:latest
```

启动容器

```
docker run --gpus all --shm-size=32g --network host \
-v ~/.cache/huggingface:/root/.cache/huggingface \
-it verlai/verl:latest bash
```

验证环境

```
python3 -c "import verl; print(verl.__version__)"
python3 -c "import vllm; print(vllm.__version__)"
python3 -c "import torch; print(torch.cuda.is_available())"
```

常见陷阱：

症状	原因	解法
`docker: command not found`	未安装 Docker	安装 Docker Desktop
`permission denied`	当前用户不在 docker 组	`sudo usermod -aG docker \$USER`
`CUDA out of memory`	显存不足	减小 micro-batch 或使用更小模型

Step 2: 跑通 quickstart（PPO @ GSM8K, 0.5B）

```
`bash
```

官方 quickstart 的核心行（见 verl docs/start/quickstart）：

```
python3 -m verl.trainer.main_ppo \
trainer.n_gpus_per_node=1 \
actor_rollout_ref.model.path=Qwen/Qwen2.5-0.5B-Instruct \
data.train_files=gsm8k/train \
data.val_files=gsm8k/test \
actor_rollout_ref.rollout.micro_batch_size=1 \
algorithm.adv_estimator=gae
```

看日志的三行（示意，非本机实录；关键词与 verl 实际日志一致，每行对应 01 章的一个手写件）：

```
[INFO] rollout.generate_sequences: Generating 256 sequences...
[INFO] sync_rollout_weights: Syncing weights from actor to rollout...
[INFO] actor.loss: Computing PPO loss with GAE advantage...
```

日志关键词	对应手写	它在干什么
`rollout` / `generate_sequences`	01 章"采样 G 个回答"	vLLM 批量生成
`sync_rollout_weights`	01 章"权重回同步"	训练权重 → 推理引擎（HybridEngine）

日志关键词	对应手写	它在干什么
`actor/...` + `critic/...` 或 `adv`	优势计算 + clip 更新	`adv_estimator=gae`(PPO) / `grpo`

验收：`val/test_score` 随 step 上升（GSM8K 准确率从基线涨几个点即成功）。

Step 3: PPO → GRPO（一处配置）

```
`bash
```

同一命令把 advantage 换成组内标准化（01 章手写的 `group_advantages`）：

```
python3 -m verl.trainer.main_ppo \
trainer.n_gpus_per_node=1 \
actor_rollout_ref.model.path=Qwen/Qwen2.5-0.5B-Instruct \
data.train_files=gsm8k/train \
data.val_files=gsm8k/test \
actor_rollout_ref.rollout.micro_batch_size=1 \
algorithm.adv_estimator=grpo \
actor_rollout_ref.rollout.n=5`
```

- `n=5` 的含义：每个 prompt 采 5 个回答（= 我们的 G；论文常用 4-16）
- 预期观察：
- GRPO 显存占用比 PPO 低（没有 critic/Value 网络——Part 8 04 章"Critic-Free"的实证）
 - 每个 prompt 的采样数 `n` 直接乘进 rollout 成本

Step 4: 自定义奖励函数（你唯一必写的代码）

quickstart 内置 GSM8K 规则奖励；换成你自己的（目录方式）：

```
`python
```

`my_reward.py` —— 语义与 Part 11 脚本 01 的 `gsm8k_reward` 相同，接口按 verl 约定

```
def compute_score(response: str, ground_truth: str) -> float:
    """verl 约定的奖励函数接口。

    Args:
        response: 模型生成的回答
        ground_truth: 标准答案

    Returns:
        float: 奖励分数（0.0 或 1.0）
    """
```

抽取逻辑与脚本 01 相同

```
import re
```

Step 1: 尝试抽取 `\boxed{}` 中的内容

```
m = re.findall(r"\boxed\{(-?[\d,\.]+)\}", response)
```

Step 2: 如果没有 `\boxed{}`, 尝试抽取 '#### 42' 格式

```
if not m:
```

```
m = re.findall(r"####s*(-?[\d,\.]+)", response)
```

Step 3: 如果都没有, 抽取最后一个数字

```
if not m:
```

```
m = re.findall(r"-?\d+\.?\d*", response.replace(", ", ""))
```

Step 4: 没有找到任何数字 → 错误

```
if not m:
```

```
return 0.0
```

Step 5: 取最后一个匹配的数字

```
pred = m[-1].replace(", ", "").rstrip(".")
```

Step 6: 比较预测值和真实值

```
try:
```

```
return 1.0 if abs(float(pred) - float(ground_truth)) < 1e-4 else 0.0
```

```
except ValueError:
```

```
return 0.0
```

练习方向（每个都是真实的 RLVR 项目形态）：

- 数学（数字对错）→ 代码（跑单测通过率）→ 格式遵循（JSON schema 校验）

△ 奖励函数是 RLVR 的最高杠杆也是最大风险点：

规则有洞（如“只看最后数字”）→ 模型学会钻洞（reward hacking, Part 8 07 章的污染近亲）。

Step 5: 双卡扩展（有 2×4090 时）

```
`bash
python3 -m verl.trainer.main_ppo \
trainer.n_gpus_per_node=2 \
trainer.nnodes=1 \
actor_rollout_ref.model.path=Qwen/Qwen2.5-0.5B-Instruct \
data.train_files=gsm8k/train \
data.val_files=gsm8k/test \
actor_rollout_ref.rollout.micro_batch_size=1 \
algorithm.adv_estimator=grpo \
actor_rollout_ref.rollout.n=5
```

verl 自动用 FSDP2 分片训练角色（Part 10 03 章的知识直接兑现）；

rollout 引擎也有 tensor-parallel 尺寸可配（`actor_rollout_ref.rollout.tensor_model_parallel_size`）

观察：

- 显存峰值下降
- step 时间未必减半（rollout 常是新瓶颈——RL 训练是生成瓶颈，这解释了为什么 verl/slime 都在 rollout 引擎上卷）

工程实践

性能分析

RL 训练的时间分布：

阶段	时间占比	瓶颈类型
rollout（生成回答）	60-80%	生成瓶颈（memory-bound）
reward（打分）	5-10%	计算瓶颈（CPU-bound）
training（更新参数）	15-30%	计算瓶颈（compute-bound）

优化方向：

- rollout：使用 vLLM/SGLang 高吞吐生成
- reward：并行打分（多进程）
- training：FSDP 分片 + 混合精度

常见陷阱

陷阱 1: OOM（显存不足）

症状：CUDA out of memory

原因：micro-batch 太大，或模型太大

解法：与[错误 4: 显存不足 (OOM)](#错误-4-显存不足-oom)同源——减小 micro-batch、换更小模型、必要时 QLoRA + 梯度检查点，完整命令见错误 4。

陷阱 2: 版本冲突

症状：ImportError: cannot import name 'xxx' from 'yyy'（或 Docker 容器启动失败）

原因：verl 与 vllm/torch/transformers 版本锁步耦合，裸 pip 装不出兼容组合

解法：

- 使用官方 Docker 镜像，不要裸 pip
- 使用 latest release tag 的官方镜像
- 遇到问题先检查版本兼容性

陷阱 3: 训练不收敛

症状：loss 不下降，或准确率不上升

原因：奖励函数有 bug，或学习率太大

解法：

- 检查奖励函数（用脚本 01 验证）

- 减小学习率（RL 阶段通常比 SFT 阶段小）
- 增加 KL 惩罚系数

最佳实践

配置调优

参数	推荐值	说明
`algorithm.adv_estimator`	`grpo`	比 PPO 更稳定
`actor_rollout_ref.rollout.n`	4-16	组大小，越大越稳定但越贵
`actor_rollout_ref.rollout.micro_batch_size`	1	4090 上防 OOM
`algorithm.kl_penalty`	0.01-0.1	防止策略偏离太远
`actor_rollout_ref.actor.lr`	1e-6 ~ 1e-5	RL 阶段学习率

日志解读

`bash`

关键指标

rollout/generate_sequences: 生成序列数
sync_rollout_weights: 权重同步时间
actor/loss: 训练损失
val/test_score: 验证分数（最重要！）

调试展示：常见错误与修复

错误 1: Docker 容器无法访问 GPU

症状：

`bash`

docker: Error response from daemon: could not select device driver "nvidia"

原因：未安装 NVIDIA Container Toolkit

解法：

`bash`

安装 NVIDIA Container Toolkit

```
distribution=$(./etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/libnvidia-container/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```



```
sudo systemctl restart docker
```

错误 2: verl 启动报错 "No module named 'vllm'"

症状：

```
`bash
```

```
ModuleNotFoundError: No module named 'vllm'
```

原因：Docker 镜像版本不对，或手动安装了错误版本

解法：

```
`bash
```

使用官方 Docker 镜像

```
docker pull verlai/verl:latest
```

或在容器内安装

```
pip install vllm==0.8.x # 版本要与 verl 兼容
```

错误 3: 训练过程中 NaN loss

症状：

```
[INFO] actor/loss: nan
```

```
[INFO] val/test_score: 0.0
```

原因：学习率太大，或奖励函数返回异常值

解法：

```
`bash
```

减小学习率

```
actor_rollout_ref.actor.lr=1e-6
```

检查奖励函数

```
python3 -c "  
from my_reward import compute_score  
print(compute_score('答案是 42', '42')) # 应该返回 1.0  
print(compute_score('我不会', '42')) # 应该返回 0.0  
"
```

错误 4: 显存不足 (OOM)

症状：

RuntimeError: CUDA out of memory. Tried to allocate 2.00 MiB

原因：micro-batch 太大，或模型太大

解法：

``bash`

减小 micro-batch

`actor_rollout_ref.rollout.micro_batch_size=1`

或使用更小的模型

`actor_rollout_ref.model.path=Qwen/Qwen2.5-0.5B-Instruct`

或使用 QLoRA 减少显存

`actor_rollout_ref.model.enable_gradient_checkpointing=true`

错误 5: 权重同步失败

症状：

`[ERROR] sync_rollout_weights: Failed to sync weights`

原因：GPU 内存不足，或 NCCL 通信问题

解法：

``bash`

检查 GPU 内存

`nvidia-smi`

使用 gloo 后端（更稳定）

`trainer.backend=gloo`

或减少 rollout 并行度

`actor_rollout_ref.rollout.tensor_model_parallel_size=1`

性能数据（量级参考）

模型	硬件	算法	组大小 n	每步时间	显存占用	验证分数
0.5B	1×4090	PPO	-	~3s	~12GB	GSM8K 20% → 35%
0.5B	1×4090	GRPO	5	~2s	~8GB	GSM8K 20% → 38%
0.5B	1×4090	GRPO	16	~5s	~12GB	GSM8K 20% → 42%
0.5B	2×4090	GRPO	5	~1.5s	~6GB/卡	GSM8K 20% → 38%

模型	硬件	算法	组大小 n	每步时间	显存占用	验证分数
7B	2×4090	GRPO	8	~30s	~20GB/卡	GSM8K 45% → 65%

- > 数据来源：官方 benchmark 与课程设计推算的量级参考（非本机实录；Docker 实操后请以自己日志为准）
- > 环境口径：本课脚本环境 torch 2.6.0+cu124；Docker 内以镜像为准
- >
- > 观察：
- > - GRPO 比 PPO 省显存（没有 critic 网络）
- > - 组大小 n 越大，效果越好但成本线性增加
- > - 双卡不一定比单卡快（rollout 是瓶颈）

之后读什么

recipe/ 目录是 verl 的"生产配方库"（DAPO=字节论文同款、GSPO、SPPO...），每个 recipe = 论文方法 + 完整可跑配置。

面试聊"你们 RL 用什么框架"时：
能说出 verl/slime 的角色分工 + HybridEngine 权重同步问题 + recipe 调优经验，
就是"上手过工业 RL"的可信信号。

学完本章你能...

- 在 Docker 里跑通 0.5B 的 PPO/GRPO 并读懂关键日志
- 写自定义奖励函数，说出 reward hacking 的风险与防范
- 解释 RL 训练中 rollout/训练双引擎与权重同步
- 配置 GRPO 的组大小 n 并评估其对成本/效果的影响
- 扩展到双卡训练并理解 FSDP 分片

课后练习

<details>
<summary>Q1: 为什么 GRPO 模式下不需要 critic 角色？显存省了多少？</summary>

A: 原因：

- 基线来自组内平均（无需学习），省掉 Value 网络

显存节省（按 Part 10 的 16Ψ 账本）：

- 一整份 7B 的训练状态（参数+梯度+优化器）≈ 112GB 不再需要
- 这就是 Part 8 04 章"Critic-Free"的工业意义

</details>
<details>
<summary>Q2: n（组大小）从 5 提到 32，成本和效果各怎么变？</summary>

A: 成本：

- rollout 成本线性 ×6.4（生成是 RL 最贵阶段）
- 显存也线性增加（需要存储更多回答）

效果：

- 优势估计更准（std 估计更稳）
- 更可能有"组内有区分度"（不会全对/全错）
- 但收益递减（从 4→16 提升大，从 16→32 提升小）

实际权衡：在 8-16 之间权衡，另配 prompt 难度过滤（全对的题跳过）

</details>

<details>

<summary>Q3: verl 的 HybridEngine 解决了什么问题？怎么验证它生效了？</summary>

A: 问题：

- rollout 和 training 用不同的引擎（vLLM vs FSDP2）
- 每次更新后要把新权重搬进推理引擎（大模型上这是 GB 级拷贝）
- 这个拷贝开销很大

解决方案：

- HybridEngine 用重分片+原地转换
- 消除 train↔rollout 转换的显存冗余

验证方法：

- 看日志里的 `sync_rollout_weights` 时间
- 应该比直接拷贝快很多

</details>

课后作业

完成本章后，去 Assignment 11 完成练习：

[Assignment 11](../../assignments/assignment_11/)

下一步

RL 的前提是好的 SFT 与数据——回 Part 12 补工具链；或去 Part 13 看数据本身怎么来。

[← 上一章](01_handwritten_to_verl.md) | [Part 11 README](README.md)